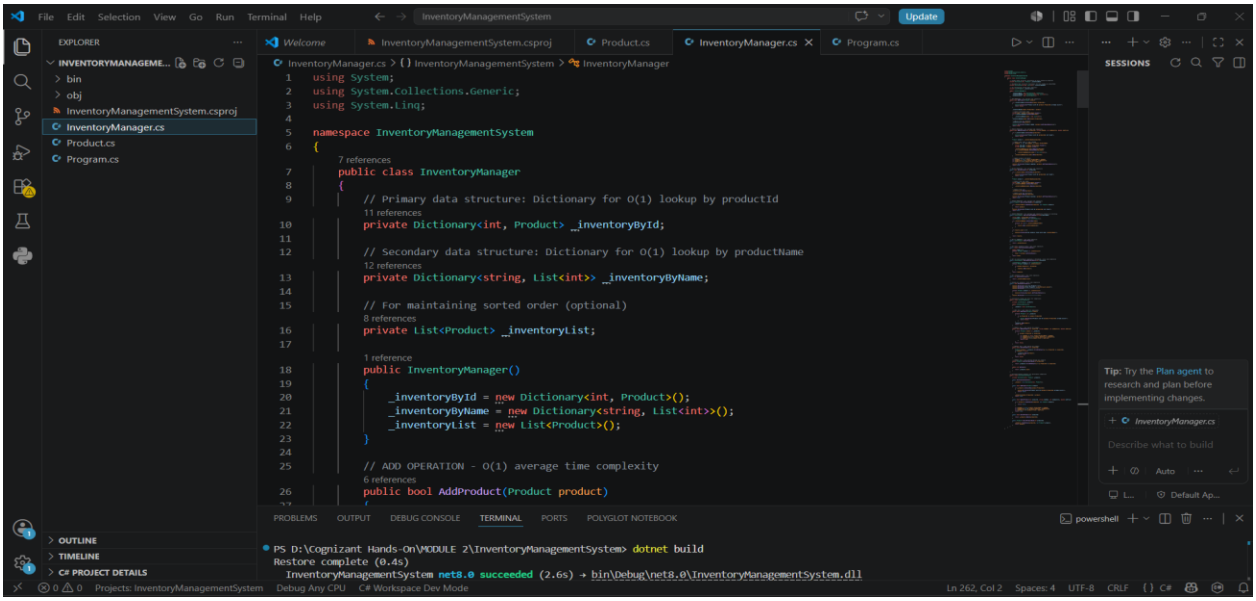
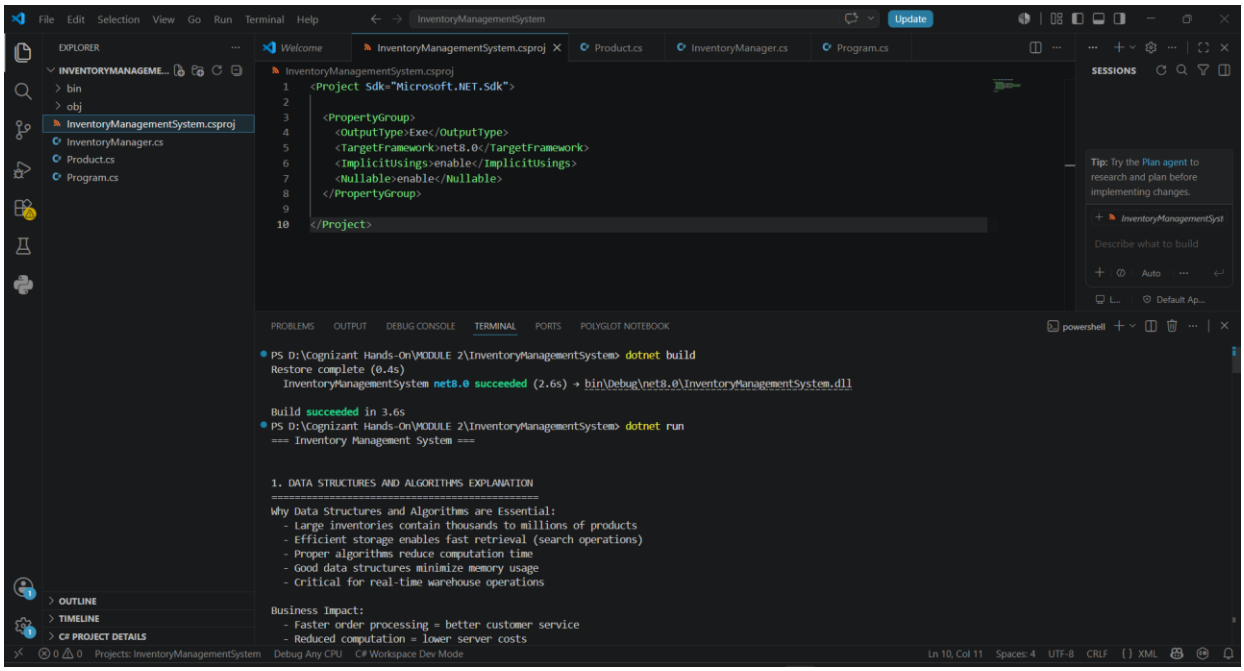
Hands-On

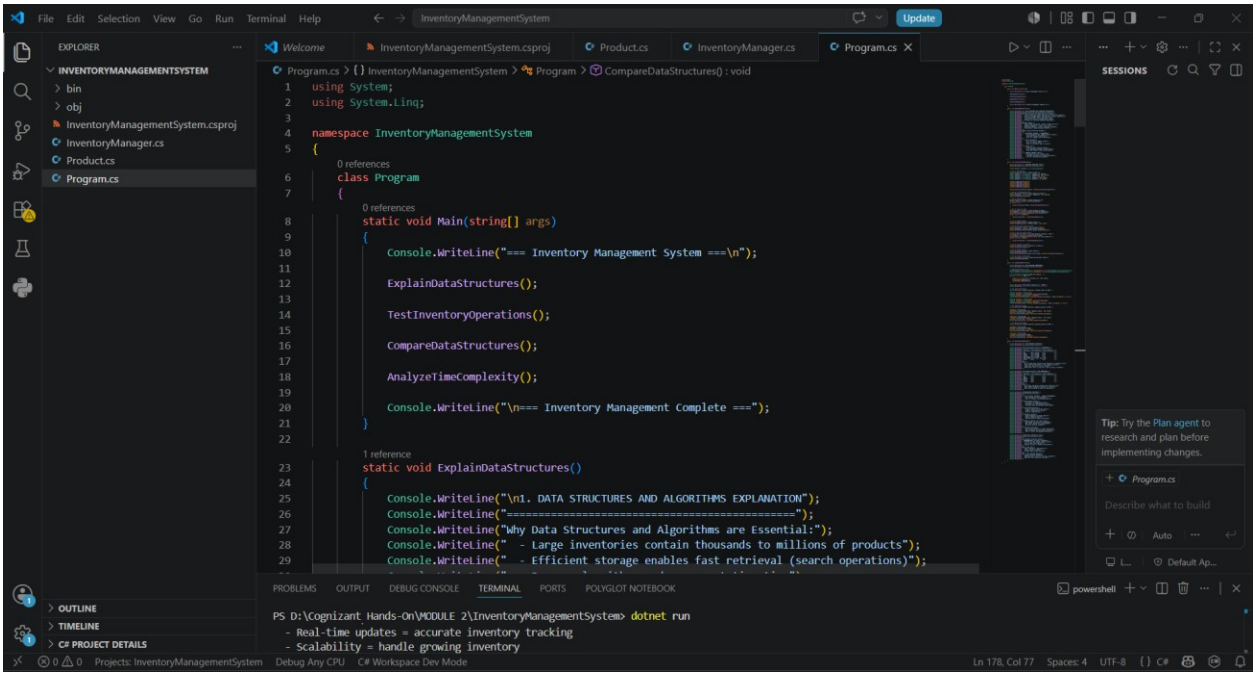
Exercise 1: Inventory Management System

During this module, I was able to successfully implement, execute and test an Inventory Management System in .NET, they have developed a project called InventoryManagementSystem. I developed a Product class that includes attributes like ProductId, ProductName, Quantity, and Price to model and handle the details of products in the inventory. Choosing a data structure like Dictionary was essential for efficient data storage and organization, enabling faster retrieval and management of product data through product IDs as keys. I then introduced techniques for adding, updating and removing products from the inventory system to carry out the necessary operations in the warehouse effectively. I also examined the time complexity of operations like add, update and delete, and noted the impact of choosing an efficient data structure on the performance of these operations. This hands-on activity was useful for me to understand the significance of data structures and algorithms in handling huge inventories, increasing the efficiency of the system, minimizing the processing time, and creating scalable and maintainable .NET applications.

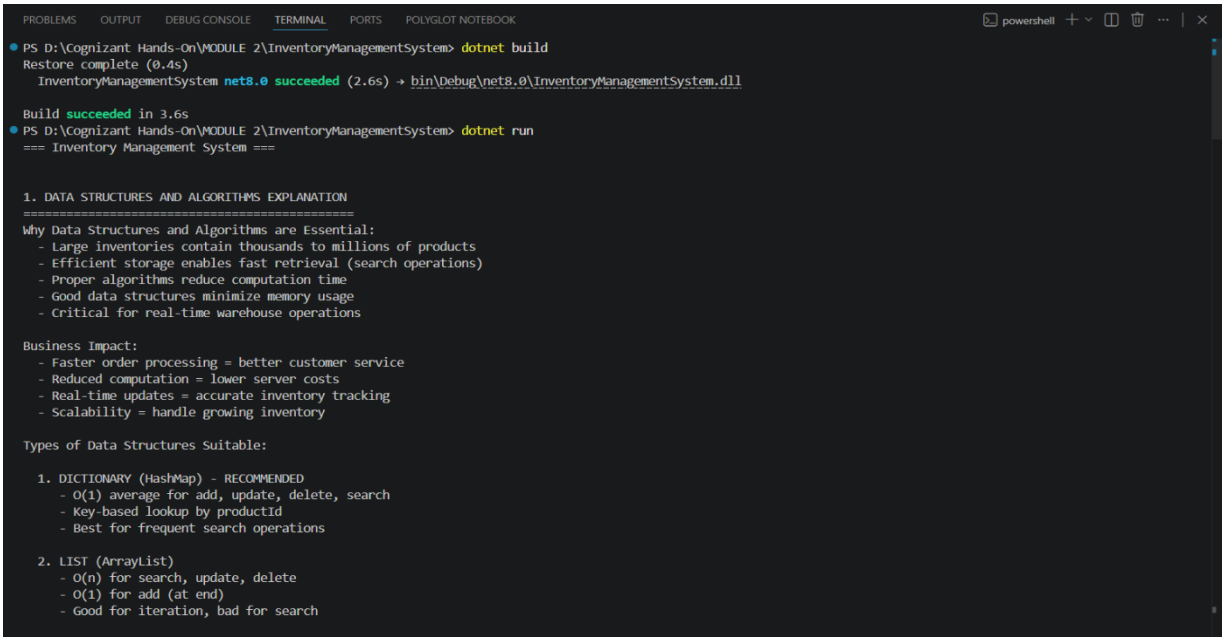
Coding:

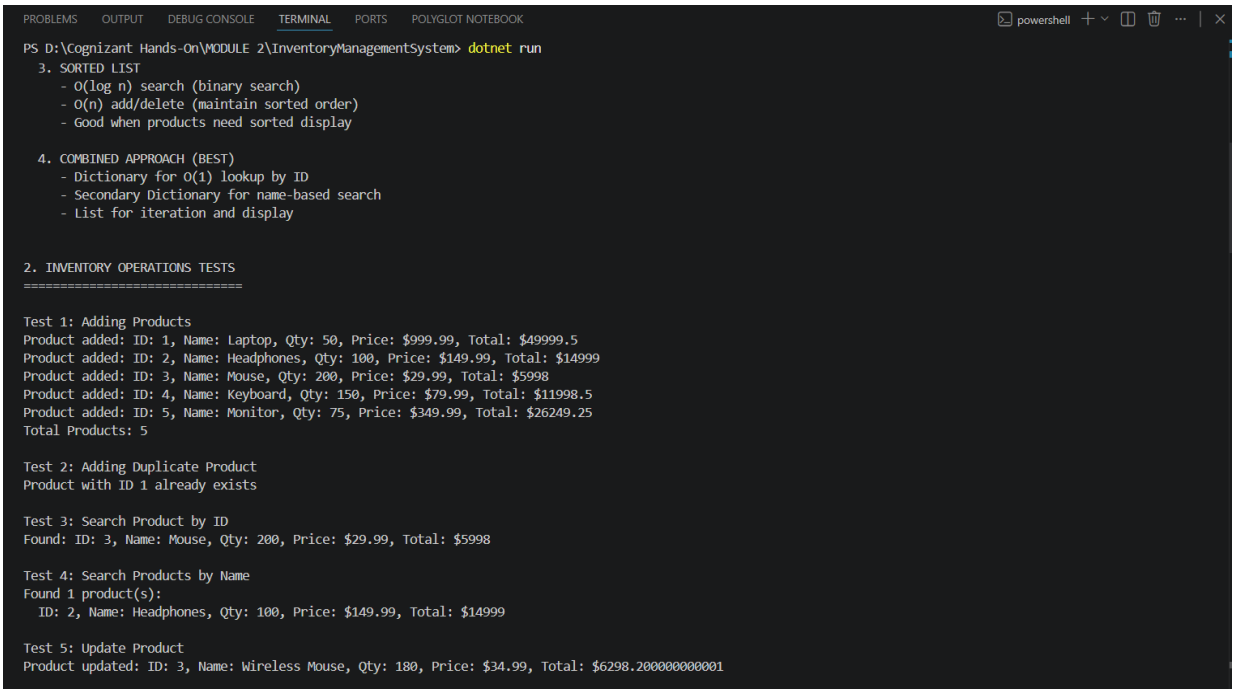



File Edit Selection View Go Run Terminal Help
InventoryManagementSystem
Update
EXPLORER
INVENTORYMANAGEME...
bin
obj
InventoryManagementSystem.csproj
InventoryManager.cs
Product.cs
Program.cs
Welcome
InventoryManagementSystem.csproj
Product.cs
InventoryManager.cs
Program.cs
InventoryManagementSystem.csproj
<Project Sdk="Microsoft.NET.Sdk">
<PropertyGroup>
<OutputType>Exe</OutputType>
<TargetFramework>net8.0</TargetFramework>
<ImplicitUsings>enable</ImplicitUsings>
<Nullable>enable</Nullable>
</PropertyGroup>
</Project>
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
Describe what to build
Auto
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet build
Restore complete (0.4s)
InventoryManagementSystem net8.0 succeeded (2.6s) → bin\Debug\net8.0\InventoryManagementSystem.dll
Build succeeded in 3.6s
PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet run
=== Inventory Management System ===
1. DATA STRUCTURES AND ALGORITHMS EXPLANATION
Why Data Structures and Algorithms are Essential:
- Large inventories contain thousands to millions of products
- Efficient storage enables fast retrieval (search operations)
- Proper algorithms reduce computation time
- Good data structures minimize memory usage
- Critical for real-time warehouse operations
Business Impact:
- Faster order processing = better customer service
- Reduced computation = lower server costs
OUTLINE
TIMELINE
C# PROJECT DETAILS
Projects: InventoryManagementSystem
Debug Any CPU
C# Workspace Dev Mode
Ln 10, Col 11
Spaces: 4
UTF-8
CRLF
XML

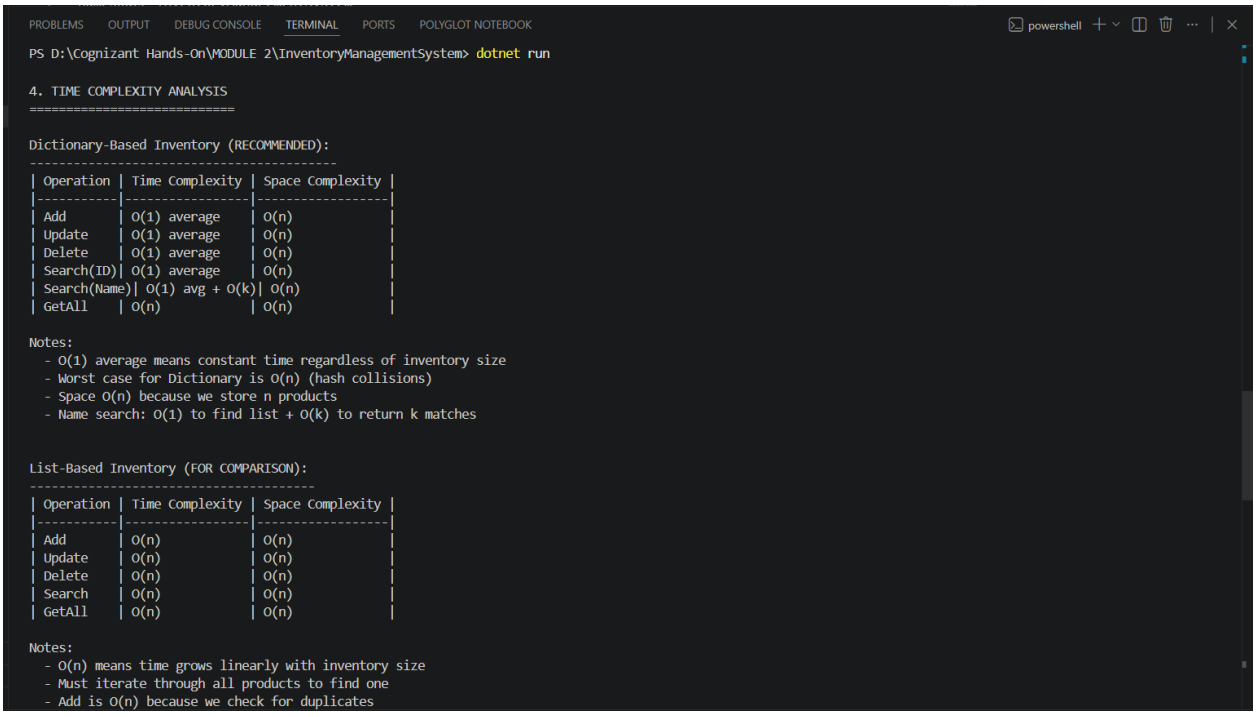

File Edit Selection View Go Run Terminal Help
InventoryManagementSystem
Update
EXPLORER
INVENTORYMANAGEMENTSYSTEM
bin
obj
InventoryManagementSystem.csproj
InventoryManager.cs
Product.cs
Program.cs
Welcome
InventoryManagementSystem.csproj
Product.cs
InventoryManager.cs
Program.cs
Program.cs > { } InventoryManagementSystem > Program > CompareDataStructures() : void
using System;
using System.Linq;
namespace InventoryManagementSystem
{
0 references
class Program
{
0 references
static void Main(string[] args)
{
Console.WriteLine("=== Inventory Management System ===\n");
ExplainDataStructures();
TestInventoryOperations();
CompareDataStructures();
AnalyzeTimeComplexity();
Console.WriteLine("\n=== Inventory Management Complete ===");
}
1 reference
static void ExplainDataStructures()
{
Console.WriteLine("\n1. DATA STRUCTURES AND ALGORITHMS EXPLANATION");
Console.WriteLine("==========");
Console.WriteLine("Why Data Structures and Algorithms are Essential:");
Console.WriteLine(" - Large inventories contain thousands to millions of products");
Console.WriteLine(" - Efficient storage enables fast retrieval (search operations)");
SESSIONS
Tip: Try the Plan agent to research and plan before implementing changes.
Program.cs
Describe what to build
Auto
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet run
- Real-time updates = accurate inventory tracking
- Scalability = handle growing inventory
OUTLINE
TIMELINE
C# PROJECT DETAILS
Projects: InventoryManagementSystem
Debug Any CPU
C# Workspace Dev Mode
Ln 178, Col 77
Spaces: 4
UTF-8
C#

Output:


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POLYGLOT NOTEBOOK
powershell
PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet build
Restore complete (0.4s)
InventoryManagementSystem net8.0 succeeded (2.6s) → bin\Debug\net8.0\InventoryManagementSystem.dll
Build succeeded in 3.6s
PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet run
=== Inventory Management System ===
1. DATA STRUCTURES AND ALGORITHMS EXPLANATION
Why Data Structures and Algorithms are Essential:
- Large inventories contain thousands to millions of products
- Efficient storage enables fast retrieval (search operations)
- Proper algorithms reduce computation time
- Good data structures minimize memory usage
- Critical for real-time warehouse operations
Business Impact:
- Faster order processing = better customer service
- Reduced computation = lower server costs
- Real-time updates = accurate inventory tracking
- Scalability = handle growing inventory
Types of Data Structures Suitable:
1. DICTIONARY (HashMap) - RECOMMENDED
- O(1) average for add, update, delete, search
- Key-based lookup by productId
- Best for frequent search operations
2. LIST (ArrayList)
- O(n) for search, update, delete
- O(1) for add (at end)
- Good for iteration, bad for search



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POLYGLOT NOTEBOOK                powershell

PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet run
  3. SORTED LIST
     - O(log n) search (binary search)
     - O(n) add/delete (maintain sorted order)
     - Good when products need sorted display

  4. COMBINED APPROACH (BEST)
     - Dictionary for O(1) lookup by ID
     - Secondary Dictionary for name-based search
     - List for iteration and display


2. INVENTORY OPERATIONS TESTS
==============================

Test 1: Adding Products
Product added: ID: 1, Name: Laptop, Qty: 50, Price: $999.99, Total: $49999.5
Product added: ID: 2, Name: Headphones, Qty: 100, Price: $149.99, Total: $14999
Product added: ID: 3, Name: Mouse, Qty: 200, Price: $29.99, Total: $5998
Product added: ID: 4, Name: Keyboard, Qty: 150, Price: $79.99, Total: $11998.5
Product added: ID: 5, Name: Monitor, Qty: 75, Price: $349.99, Total: $26249.25
Total Products: 5

Test 2: Adding Duplicate Product
Product with ID 1 already exists

Test 3: Search Product by ID
Found: ID: 3, Name: Mouse, Qty: 200, Price: $29.99, Total: $5998

Test 4: Search Products by Name
Found 1 product(s):
  ID: 2, Name: Headphones, Qty: 100, Price: $149.99, Total: $14999

Test 5: Update Product
Product updated: ID: 3, Name: Wireless Mouse, Qty: 180, Price: $34.99, Total: $6298.200000000001
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POLYGLOT NOTEBOOK                powershell

PS D:\Cognizant Hands-On\MODULE 2\InventoryManagementSystem> dotnet run

4. TIME COMPLEXITY ANALYSIS
============================

Dictionary-Based Inventory (RECOMMENDED):
----------------------------------------
| Operation | Time Complexity | Space Complexity |
|-----------|-----------------|------------------|
| Add       | O(1) average    | O(n)             |
| Update    | O(1) average    | O(n)             |
| Delete    | O(1) average    | O(n)             |
| Search(ID)| O(1) average    | O(n)             |
| Search(Name)| O(1) avg + O(k)| O(n)            |
| GetAll    | O(n)            | O(n)             |

Notes:
  - O(1) average means constant time regardless of inventory size
  - Worst case for Dictionary is O(n) (hash collisions)
  - Space O(n) because we store n products
  - Name search: O(1) to find list + O(k) to return k matches


List-Based Inventory (FOR COMPARISON):
--------------------------------------
| Operation | Time Complexity | Space Complexity |
|-----------|-----------------|------------------|
| Add       | O(n)            | O(n)             |
| Update    | O(n)            | O(n)             |
| Delete    | O(n)            | O(n)             |
| Search    | O(n)            | O(n)             |
| GetAll    | O(n)            | O(n)             |

Notes:
  - O(n) means time grows linearly with inventory size
  - Must iterate through all products to find one
  - Add is O(n) because we check for duplicates
```